SOURCECODE DOCUMENTATION FOR THE F1 APP

Table Of Contents

DB.PHP 2
ACTION.PHP 6
F1-FRONTEND.HTML 8
SCRIPT.JS 10

DB.PHP

The db.php file is responsible for establishing the connection between the F1 Ticket Booking System and the MySQL database. It also contains the main database functions used in this project, such as inserting booking data, checking whether the bookings table exists, creating the table if required, and fetching booking records.

```php
<?php
define('DB_SERVER', 'f1tickets.cntbvydiscc4.us-east-1.rds.amazonaws.com');
define('DB_USERNAME', 'admin');
define('DB_PASSWORD', 'Password');
define('DB_DATABASE', 'f1tickets');
```

Here we are defining the database host address, username, password and name.
DB_SERVER defines the database host address
DB_USERNAME defines the database username
DB_PASSWORD defines the database password
DB_DATABASE defines the database name.

```php
/* Connect to MySQL and Select Database */
$database = mysqli_connect(DB_SERVER, DB_USERNAME, DB_PASSWORD, DB_DATABASE);
```

MySQLi (MySQL Improved) is a PHP extension used to connect and interact with MySQL databases.
Here we are establishing a database connection using mysqli_connect() function. We are storing the database credentials in the function .

```php
if (!$database) {
    echo json_encode(array(
        'message' => "Failed to connect to MySQL: " . mysqli_connect_error(),
        'status' => 'error'
    ));
    die();
}
```

The system will try connecting it to the database, if it fails connecting we print an error message in JSON format and terminate the script.

```php
/* INSERT BOOKING */
function addContest($data) {
    global $database;

    $name = mysqli_real_escape_string($database, $data['name']);
    $email = mysqli_real_escape_string($database, $data['email']);
    $race = mysqli_real_escape_string($database, $data['race']);
    $ticketTier = mysqli_real_escape_string($database, $data['ticketTier']);

    $query = "INSERT INTO BOOKINGS (name, email, race, ticketTier)
              VALUES ('$name', '$email', '$race', '$ticketTier');";

    if (!mysqli_query($database, $query)) {
        echo json_encode(array(
            'message' => 'Error saving booking',
            'status' => 'error',
            'sql_error' => mysqli_error($database)
        ));
        die();
    } else {
        echo json_encode(array(
            'message' => 'Booking successful!',
            'status' => 'success'
        ));
        die();
    }
}
```

We are using this function to insert a booking into the database mainly in the Bookings table. Firstly, we give the function access to the main database connection using global $database
Then we sanitize the input values using mysqli_real_escape_string()
Then we create a SQL query and execute it using mysqli_query() and finally return a JSON success or error response based on the result.

```php
/* CREATE TABLE IF NOT EXISTS */
function verifyContestTable() {
    global $database;
    $table = 'BOOKINGS';

    if (!isTableExists($table)) {
        $query = "CREATE TABLE $table (
            id INT(11) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100),
            email VARCHAR(100),
            race VARCHAR(255),
            ticketTier VARCHAR(100),
            created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        );";

        if (!mysqli_query($database, $query)) {
            echo json_encode(array(
                'message' => 'Error creating table!',
                'status' => 'error',
                'sql_error' => mysqli_error($database)
            ));
            die();
        }
    }
}
```

This function checks whether the Booking table exists. It sets the table name as Bookings. If the table does not exist, it creates it using CREATE TABLE . Adds the columns and stores the data in the table.

```php
/* CHECK TABLE EXISTS */
function isTableExists($tableName) {
    global $database; // Changed from $connection to $database

    $t = mysqli_real_escape_string($database, $tableName);
    $d = mysqli_real_escape_string($database, DB_DATABASE);

    $checktable = mysqli_query(
        $database,
        "SELECT TABLE_NAME FROM information_schema.TABLES
         WHERE TABLE_NAME = '$t' AND TABLE_SCHEMA = '$d'"
    );

    return mysqli_num_rows($checktable) > 0;
}
```

This function checks whether the table is already present in the selected database.
First, the table name and database name are cleaned. Then the function queries information_schema.TABLES. If one or more rows are returned, the table exists. It returns a boolean result, which is then used by verifyContestTable()

```php
/* FETCH BOOKINGS */
function getContest() {
    global $database;
    $data = array();
    $query = "SELECT * FROM BOOKINGS ORDER BY id DESC";

    if ($result = mysqli_query($database, $query)) {
        while ($row = mysqli_fetch_assoc($result)) {
            $data[] = $row;
        }
        mysqli_free_result($result);
    }

    return $data;
}
?>
```

This function is used to fetch all booking records. It accesses the database connection. Runs SELECT * FROM BOOKINGS ORDER BY id DESC";. It stores each in an array and returns the array. It shows the latest booking first.

ACTION.PHP

The action.php file is responsible for handling requests that come from the frontend. It acts as a request handler of the F1 Ticket Booking System. It receives data from the frontend, validates the input, and performs the required operation such as storing or retrieving booking records.

```php
<?php
include('./db.php');
```

We start by including dp.php, this will give access to the database connection and helping functions required for database operations.

```php
// 1. ADD BOOKING (POST REQUEST)
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action']) && $_POST['action'] === 'contest') {
```

Here we are checking three things:
Request method is POST
ACTION exists in $_POST
ACTION value is CONTEST
Only if all three are true then we run the code

```php
    // Ensure table exists before inserting
    verifyContestTable();
```

We check whether the Bookings table is there or not by calling the verifyContestTable(). If it does not exist db.php will create it.

```php
    // Collect data safely
    $data = array();
    $data['name'] = isset($_POST['name']) ? htmlentities($_POST['name']) : '';
    $data['email'] = isset($_POST['email']) ? htmlentities($_POST['email']) : '';
    $data['race'] = isset($_POST['race']) ? htmlentities($_POST['race']) : '';
    $data['ticketTier'] = isset($_POST['ticketTier']) ? htmlentities($_POST['ticketTier']) : '';
```

This part of the code collects the input from the user and puts it into an array. The input contains name, email, race, ticketTier. isset() checks if the value exists in the form submission. Then htmlentities() converts special HTML characters into safe text, which helps prevent harmful script input.

```php
    // Validate inputs
    if (empty($data['name']) || empty($data['email']) || empty($data['race']) || empty($data['ticketTier'])) {
        echo json_encode(array(
            'message' => 'Please fill all fields',
            'status' => 'error'
        ));
        exit;
    }
```

This part of the code checks whether there is any entry missing from the user. If there is then it returns an error message.

```php
        // Save booking
        addContest($data);
    }
```

Once the input is successful, the data is passed to the addContest() function in db.php for insertion in the database.

```php
// 2. FETCH BOOKINGS (GET REQUEST)
if ($_SERVER['REQUEST_METHOD'] === 'GET' && isset($_GET['action']) && $_GET['action'] === 'contest') {
```

GET is used when the system needs to retrieve data and not store it.
This will check 3 things:
Request method is GET
ACTION exists in $_GET
ACTION = CONTEST
If all of these are true, then we fetch the booking data.

```php
    // Added Fix: Ensure table exists before we try to SELECT from it
    verifyContestTable();
```

Before fetching records, the system again checks that the table exists to avoid errors during data fetching.

```php
    $contest = getContest();
    echo json_encode($contest);
    exit;
}
?>
```

The getContest() function is used to fetch all booking records, convert them into JSON and send them back to the frontend. The results are encoded in JSON format because JSON is easy for frontend JavaScript to read and display.

F1-FRONTEND.HTML

- **Overview**

  The file f1.html is a HTML/CSS file for the front-end of our F1 Ticket Booking System. It uses HTML5 and CSS 3, the main job of this file is to collect the data from the user safely and provide an empty structure where our Java Script can inject live dataset records.

1. Document Setup & Viewport Scaling (The <head>)

There are many different types of head elements which we have used in this project, like for configuration tags we used <meta>, it is used to control browser scaling and character sets, we also used <link> for external resource tags, it is used to establish secure connections to third-party CDNs, and <title> is a meta data tag, used for tab identification and basic SEO.

We also made the site mobile responsive using the following code:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

In this code we used <meta> tag to instruct the browser on how to control the site and the line “width=device width” tells the browser to set the width of the page according to the device.

We also made the use of external resource for getting the font from google:

```
<link href="https://fonts.googleapis.com/css2?family=Russo+One&display=swap" rel="stylesheet">
```

This line of code directly connects to CDN hosted by Google to download the “Russo One” font file.

2. CSS Design & Accessibility (The <style> Block)

There are many different types of techniques used to build the UI without the framework for CSS; frameworks such as Selectors they Element selectors (body), Class selectors (.section), and

Pseudo-elements (::before) , we also used Layout Properties like Flexbox (display: flex, flex-wrap, justify-content) for fluid alignment , Visual/Aesthetic properties like gradients (linear-gradient), opacity (rgba), and filters (backdrop-filter) and Positioning properties like fixed positioning (position: fixed, z-index) for background locking and layering.

Background Overlays:

```
body::before {
    content: "";
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.5);
    z-index: -1;
}
```

As you can see in this code that the body has a fixed background of and F1 car , so the command (::before) creates a pseudo-element, then we gave a black background with 505 optical (rgba(0,0,0,0.5)) and the (z-index : -1) pushes the darker layer behind the text but in front of the background image.

We also applied the CSS Flexbox module to the container holding the countdown timer. (flex-wrap: wrap) is the crucial property.

```
.timer-container {
    display: flex;
    justify-content: center;
    gap: 15px;
    flex-wrap: wrap;
}
```

We also used Glass morphism, it was applied to the main content panels, we gave it a background of 10% opacity, and we also applied blur to whichever part is directly behind the panel.

```
.section {
    background: rgba(255, 255, 255, 0.1);
    backdrop-filter: blur(10px);
    border-radius: 15px;
    padding: 30px;
    margin-bottom: 30px;
    border: 1px solid rgba(255, 255, 255, 0.2);
    box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
}
```

3. HTML Structure & Logic Integration (The <body>)

The structure and body of our html rely on a specific category of HTML tags such as Semantic Structure (<header>, <footer>, ,) for logical grouping, Form controls (<form>, <input> (Text, Email), <select>, <option>, <button>) for secure data collection , Data presentation (<table>, <thead>, <tbody>, <tr>, <th>, <td>) for organizing cloud data and Logic integration (<script>) for connecting the frontend to backend .

```
<label>Full Name:</label>
<input type="text" id="customerName" placeholder="Enter your full name" required>

<label>Email Address:</label>
<input type="email" id="email" placeholder="example@gmail.com" required>
```

In the above code by adding required attributes, the browser itself would refuse to submit if it's not filled and for the email it won't allow you to submit if there is no '@' in the mail.

```
<button type="button" id="submitBtn" onclick="addBooking()">CONFIRM BOOKING</button>
```

In the above code we use (type='button') because it adds an inline Java Script event listener (onclick= "addBooking()").

```
<script src="script.js"></script>
```

The above code connects HTML to JavaScript .

SCRIPT.JS

The **script.js** file deals with dynamic user interface including the computation of a real-time Rolex countdown timer, as well as, asynchronous communication with the server. It confirms the user input and dispatches or receives booking data via the Fetch API without having to refresh the page so that it offers a single-page application experience to users.

1. Countdown Logic (Time Arithmetic) Rolex.
This script works out the difference between the time and the season finale in 2026 in milliseconds. It also has a set interval to update the math after each 1000ms, so that the UI remains lively. The raw time is then transformed to integers (days, hours, etc) using Math.floor

and a ternary operator is used to put leading zeros after the integer to ensure a high quality Rolex clock look.

```javascript
// Set the date we're counting down to (F1 2026 Season Finale - Estimated)
const countDownDate = new Date("Dec 6, 2026 15:00:00").getTime();

// Update the countdown every 1 second
const timerInterval = setInterval(function() {

    // Get today's date and time
    const now = new Date().getTime();

    // Find the distance between now and the countdown date
    const distance = countDownDate - now;

    // Time calculations for days, hours, minutes and seconds
    const days = Math.floor(distance / (1000 * 60 * 60 * 24));
    const hours = Math.floor((distance % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60));
    const minutes = Math.floor((distance % (1000 * 60 * 60)) / (1000 * 60));
    const seconds = Math.floor((distance % (1000 * 60)) / 1000);

    // Display the result in the corresponding HTML elements
    // We add a "0" in front of single digit numbers for a cleaner clock look
    document.getElementById("days").innerHTML = days < 10 ? "0" + days : days;
    document.getElementById("hours").innerHTML = hours < 10 ? "0" + hours : hours;
    document.getElementById("minutes").innerHTML = minutes < 10 ? "0" + minutes : minutes;
    document.getElementById("seconds").innerHTML = seconds < 10 ? "0" + seconds : seconds;

    // If the countdown is finished, write some text
    if (distance < 0) {
        clearInterval(timerInterval);
        document.getElementById("rolexTimer").innerHTML = "<h3 style='color: #A37E2C;'>SEASON CONCLUDED</h3>";
    }
}, 1000);
```

2. The Fetch API (Asynchronous Data Exchange)

The system relies on the use of the fetch API to interact with the action.php without the reload of the page. With a "Promise-based" approach (.then), the application can be interactive as it waits for the database to respond. The exchange of data is conducted in the format of JSON that is considered the industry standard of lightweight and structured communication between JavaScript and PHP.

```javascript
// 3. FETCH ALL BOOKINGS FROM DATABASE
function loadBookings() {
    // We call action.php with a GET request to 'contest'
    fetch("action.php?action=contest")
    .then(response => {
        if (!response.ok) throw new Error('Network response was not ok');
        return response.json();
    })
    .then(data => {
        let tableBody = document.getElementById("bookingTableBody");
        tableBody.innerHTML = ""; // Clear existing table rows

        if (Array.isArray(data)) {
            data.forEach(booking => {
                renderTableRow(booking.name, booking.email, booking.race, booking.ticketTier);
            });
        }
    })
    .catch(error => {
        console.error('Error loading existing bookings:', error);
    });
}
```

3. Validation of data and manipulating the Document Object Model.

The script implements the Pit Stop check with the help of .trim() and email validation before sending data to the server to avoid entries of junk data. After a successful confirmation by the server, the renderTableRow helper function will manually insert a new <tr> into the table. This offers Optimistic UI, which offers the user immediate visual feedback on their booking.

```javascript
// 3. ADD NEW BOOKING
function addBooking() {
    let name = document.getElementById("customerName").value.trim();
    let email = document.getElementById("email").value.trim();
    let race = document.getElementById("race").value;
    let ticketTier = document.getElementById("ticketTier").value;

    // Validation
    if (name === "" || email === "" || race === "" || ticketTier === "") {
        alert("Please fill all fields.");
        return;
    }

    if (!email.includes("@") || !email.includes(".")) {
        alert("Please enter a valid email address.");
        return;
    }

    // Prepare data for PHP
    let formData = new FormData();
    formData.append("name", name);
    formData.append("email", email);
    formData.append("race", race);
    formData.append("ticketTier", ticketTier);
    formData.append("action", "contest");

    // Send POST request
    fetch("action.php", {
        method: "POST",
        body: formData
    })
    .then(response => response.json())
    .then(data => {
        alert(data.message);

        if (data.status === "success") {
            // Add the new row to the table immediately
            renderTableRow(name, email, race, ticketTier);

            // Clear the form for the next user
            document.getElementById("customerName").value = "";
            document.getElementById("email").value = "";
            document.getElementById("race").selectedIndex = 0;
            document.getElementById("ticketTier").selectedIndex = 0;
        }
    })
    .catch(error => {
        console.error('Error submitting booking:', error);
        alert("There was an error saving your booking. Please check console.");
    });
}
```

```
// 4. HELPER FUNCTION TO DRAW TABLE ROWS
function renderTableRow(name, email, race, ticketTier) {
    let tableBody = document.getElementById("bookingTableBody");
    let newRow = tableBody.insertRow();

    newRow.insertCell(0).innerHTML = name;
    newRow.insertCell(1).innerHTML = email;
    newRow.insertCell(2).innerHTML = race;
    newRow.insertCell(3).innerHTML = ticketTier;
}
```

4. Onloading (The window.onload Trigger)

This operation serves as a "Race Start" signal, and as such will only run data-fetching logic when the browser has completed construction of the HTML structure . By trying to execute the script immediately, it would not find the table element. The window.onload will be used to assure that the Stage is prepared (HTML) before the Actors (Database Records) are invited to do so.